

Lab 2 Submission

Agile Backlog Creation & Sprint Simulation in Jira

Problem Statement #32 — Personal Subscription & Recurring Expense Auditor

Jira Project / Workspace Link:

<https://pes1ug24cs068.atlassian.net/jira/software/projects/PSREA/boards/2?filter=&groupBy=epic>

Jira key: PSREA

Two sprints completed: PSREA Sprint 1 (26 pts) and PSREA Sprint 2 (26 pts)

Annamalai N – PES1UG24CS068

1. Jira Backlog — Epics, User Stories & Story Points

Backlog view showing PSREA Sprint 1 and the general Backlog — each user story tagged with its parent Epic and Fibonacci story point estimate. Five Epics span the project: Transaction Import & Recurring Detection, Renewal Alerts & Notifications, Cancellation Assistance, Audit & Manual Reclassification, and Security & Data Privacy.

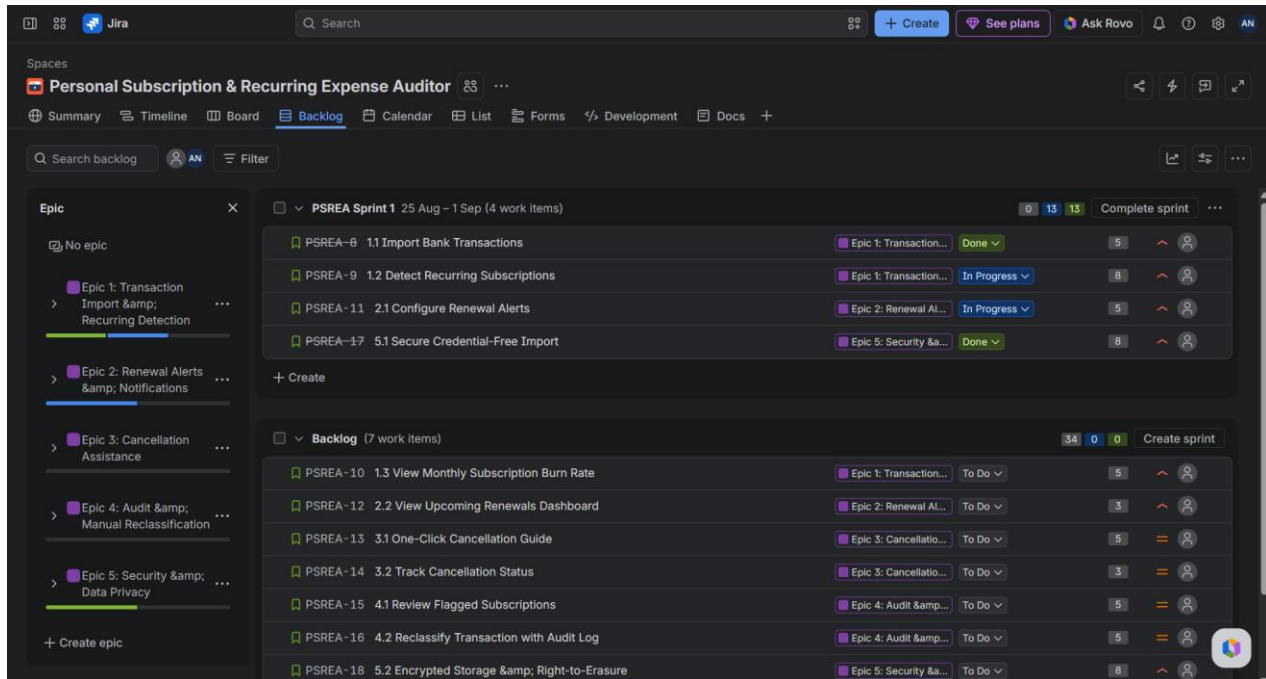


Fig. 1 — Backlog with Epics, User Stories, and story point assignments.

2. Sprint 1 — Active Sprint Board

Sprint 1 board view (grouped by Epic), showing stories progressing through To Do / In Progress / Done as work was simulated across Epic 1 (Transaction Import & Recurring Detection) and Epic 2 (Renewal Alerts & Notifications).

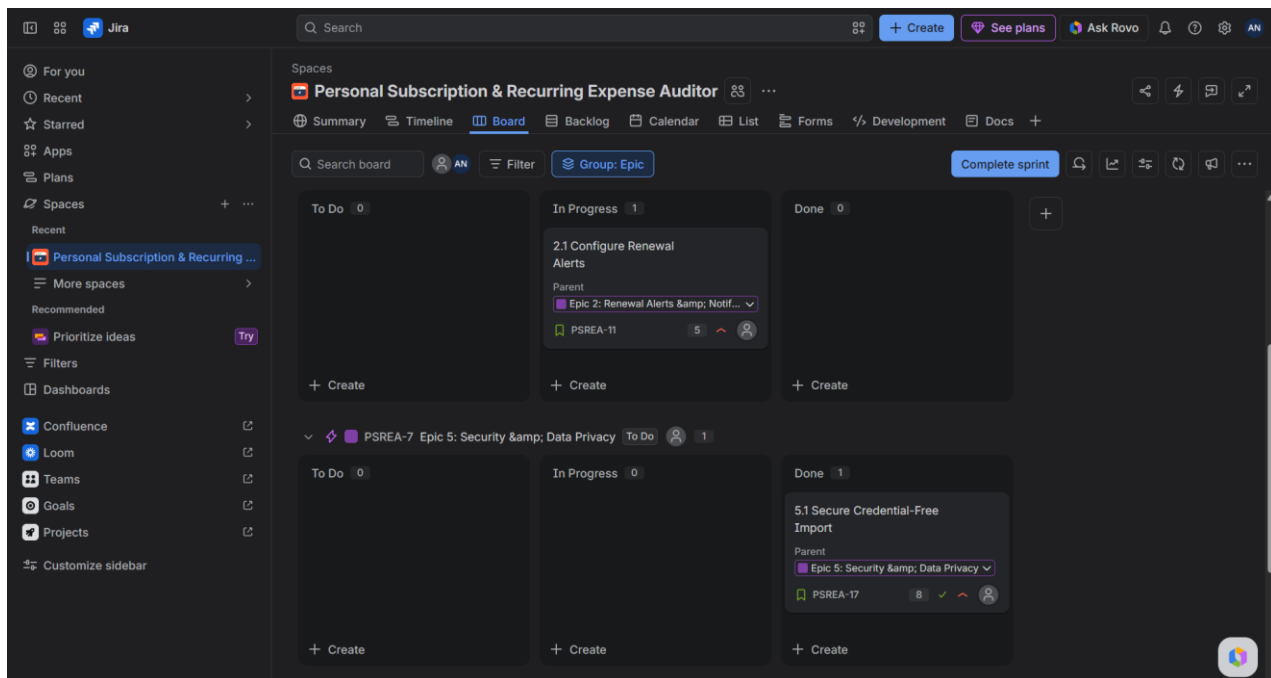
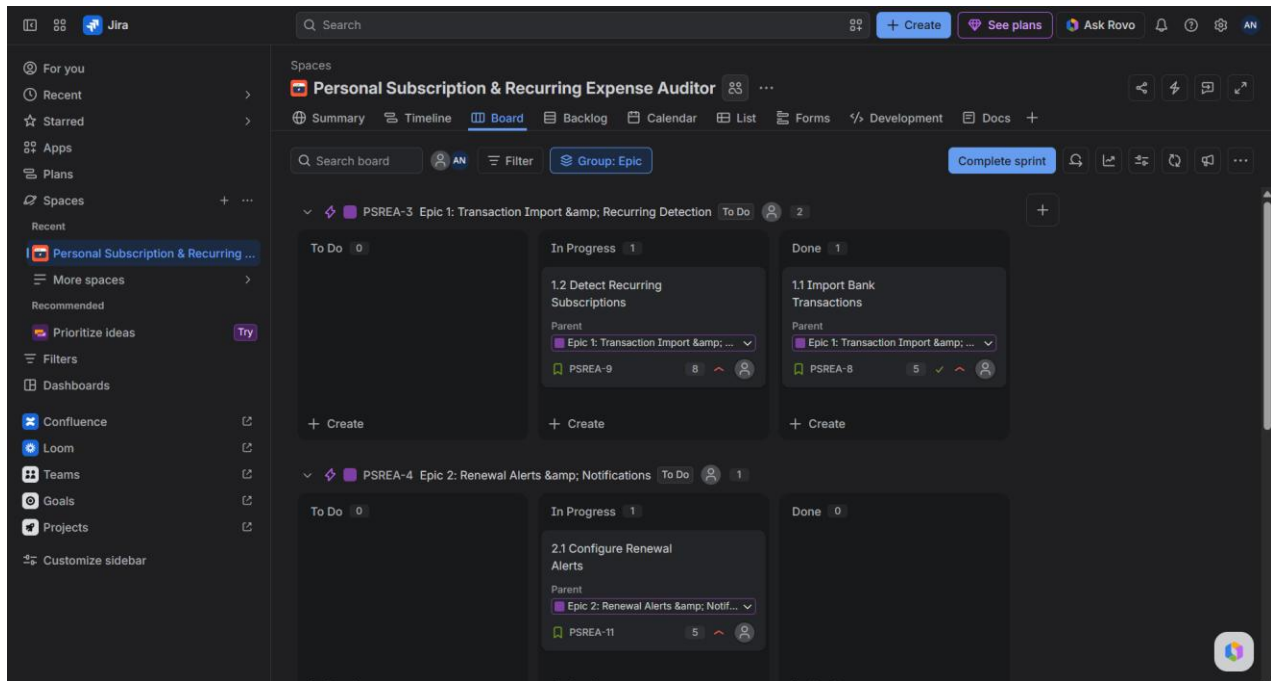


Fig. 2 — Sprint 1 board view, grouped by Epic.

3. Sprint 1 Burndown Chart

Burndown chart for PSREA Sprint 1, tracking remaining story points against the ideal guideline burn rate over the sprint window (25 Aug – 1 Sep 2026). Remaining work dropped from 26 to 13 points as two of the four committed stories reached Done.

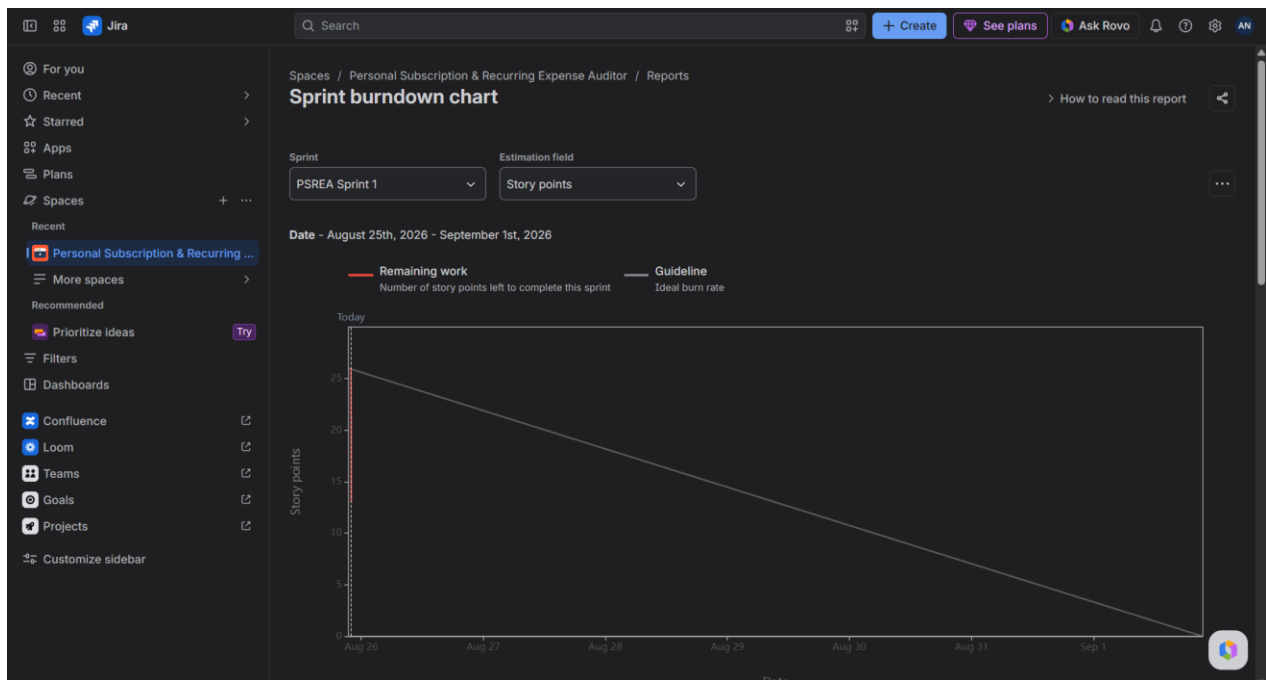


Fig. 3 — Sprint 1 burndown chart (Story Points vs. time).

4. Sprint 2 — Active Sprint Board

Sprint 2 board view (grouped by Epic), showing stories from Epic 1 (View Monthly Subscription Burn Rate) and Epic 2 (View Upcoming Renewals Dashboard) progressing through the workflow.

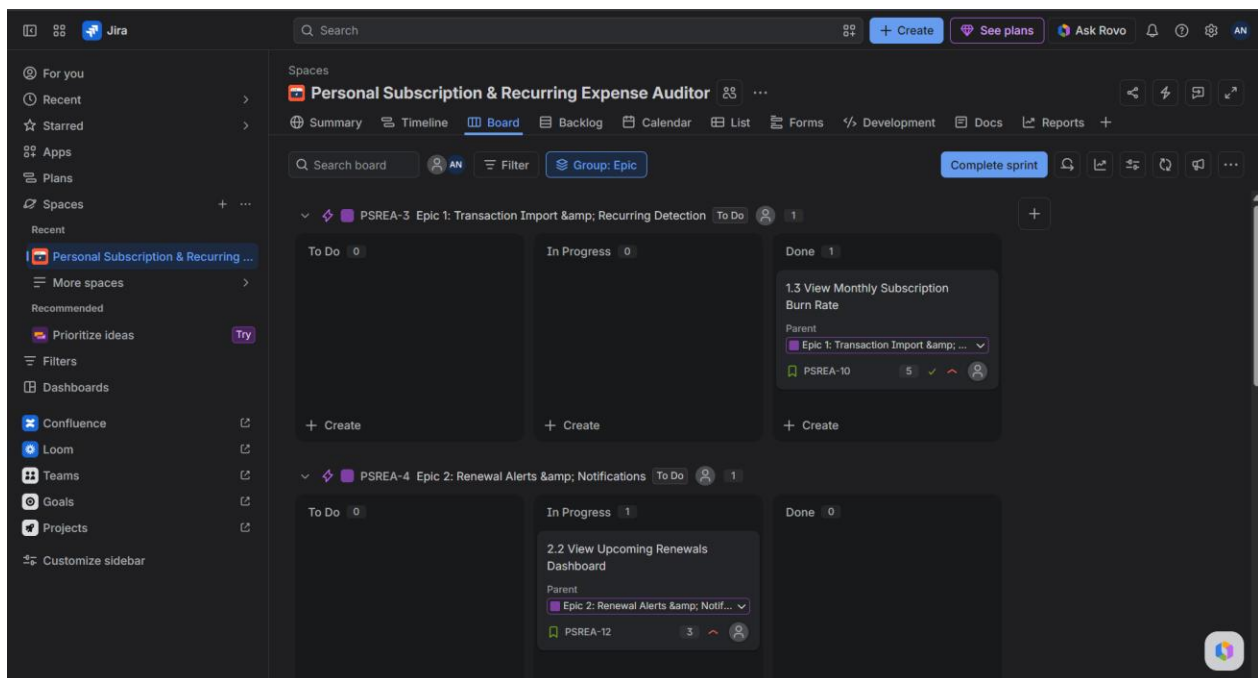


Fig. 4 — Sprint 2 board view, grouped by Epic.

5. Sprint 2 Burndown Chart

Burndown chart for PSREA Sprint 2, again over the 25 Aug – 1 Sep 2026 window, showing remaining work dropping from 26 to 13 points as two of the five committed stories (including the encryption-related story from Epic 5) were completed.



Fig. 5 — Sprint 2 burndown chart (Story Points vs. time).

6. Reflection Questions

Did the story point estimations reflect the actual effort required?

Broadly yes. The Fibonacci estimates tracked the underlying complexity of each story reasonably well. Stories involving a single, well-scoped UI or configuration surface — viewing the burn-rate dashboard (PSREA-10, 5 pts), configuring renewal alerts (PSREA-11, 5 pts), viewing upcoming renewals (PSREA-12, 3 pts) — sat at the lower end, since they mostly involve querying and displaying already-modelled data. Stories that required inference or cross-referencing, such as detecting recurring subscriptions from raw transaction history (PSREA-9, 8 pts) and reviewing flagged subscriptions against an audit trail (PSREA-15, 5 pts), were estimated higher, reflecting the extra logic needed to distinguish genuine recurring charges from one-off purchases.

The two 8-point outliers — Secure Credential-Free Import (PSREA-17) and Encrypted Storage & Right-to-Erase (PSREA-18), both under Epic 5: Security & Data Privacy — were deliberately estimated at the top of the scale, since they combine a functional requirement (letting a user import data without ever handing over bank credentials) with a genuinely cross-cutting non-functional concern (encryption at rest and a right-to-erase delete flow). In hindsight, PSREA-18 in particular could have been split into a storage-encryption story and a separate erasure-flow story, which would likely have produced two cleaner 5-point stories instead of one 8-point story that bundles two different kinds of work.

As with the estimation exercise for the other project done earlier in this lab, there's no independent effort log to validate the estimates against, since this was a simulated solo sprint rather than a real multi-person team tracking hours.

Was the backlog well-prioritized?

Yes — prioritization followed the natural dependency chain of the app rather than an arbitrary ordering. Epic 1 (Transaction Import & Recurring Detection) was scheduled first because every other epic depends on transactions actually being imported and classified — there is nothing to alert on, cancel, or audit until recurring charges have been detected. Epic 5 (Security & Data Privacy) was deliberately pulled in early alongside Epic 1 rather than left for last, since both of its stories (credential-free import and encrypted storage) are foundational to how transaction data is ingested and stored in the first place — bolting security on after the import pipeline already exists would have been the riskier order.

Epic 2 (Renewal Alerts & Notifications) followed next, since alerting on an upcoming renewal only makes sense once recurring subscriptions are already being detected. Epics 3 (Cancellation Assistance) and 4 (Audit & Manual Reclassification) were correctly left later in the backlog: cancellation guidance is only useful once a user has been alerted to a renewal they want to avoid, and manual reclassification is a correction mechanism that matters most once there's a meaningful volume of auto-detected data to review. Priority levels (7 High, 4 Medium) assigned per the requirements table were respected in scheduling — the High-priority stories from Epics 1, 2, and 5 were the ones pulled into Sprint 1 and Sprint 2, while Medium-priority stories from Epics 3 and 4 sat later in the backlog.

How did the simulated sprint align with the original plan?

Sprint 1 (planned 4 items / 26 points: PSREA-8, PSREA-9, PSREA-11, PSREA-17) closed with 2 of 4 stories fully Done (PSREA-8 "Import Bank Transactions" and PSREA-17 "Secure Credential-Free Import" — 13 of 26 points, 50%), while PSREA-9 ("Detect Recurring Subscriptions") and PSREA-11 ("Configure Renewal Alerts") were still In Progress at sprint completion and were rolled back into the Backlog. This is a reasonable outcome for a first sprint: the two stories that finished were both entry-point stories (importing data, securing the import), while the two that slipped were the stories one layer downstream — detection logic that consumes the imported data, and an alert-configuration flow that assumes subscriptions have already been detected.

Sprint 2 (planned 5 items / 26 points: PSREA-10, PSREA-12, PSREA-13, PSREA-15, PSREA-18) followed a similar shape: PSREA-10 ("View Monthly Subscription Burn Rate") and PSREA-18 ("Encrypted Storage & Right-to-Erasure") reached Done (13 of 26 points, 50%), while PSREA-12 ("View Upcoming Renewals Dashboard") and PSREA-13 ("One-Click Cancellation Guide") remained In Progress and PSREA-15 ("Review Flagged Subscriptions") stayed in To Do; all three were rolled back into the Backlog on completion. As with the other project simulated earlier, status transitions were applied programmatically in a single sitting rather than emerging from day-by-day work, so the **sequence** of what finished first (foundational, security- and data-facing stories) is meaningful, while the **timing** is compressed relative to a real sprint.

What insights did the burndown chart give about team capacity?

Both burndown charts show the same pattern: a single near-vertical drop from 26 remaining points down to 13, rather than the gradual day-by-day staircase a real sprint would produce. As with the reflection for the other project, the honest read is that this is an artifact of the simulation method — all transitions were applied via the API in one batch instead of being logged incrementally over the sprint window. In a real team, a burndown that flat for most of the sprint and then drops all at once would be worth raising in a retrospective, since it usually signals that progress isn't being reported until very late.

Setting that artifact aside, the more durable signal from both charts is the identical 50% completion rate: in both sprints, exactly half of the committed points reached Done by the time the sprint was closed, with the other half rolling into the Backlog. That's a fairly consistent (if modest) velocity signal — two data points both landing at 13 of 26 points suggests the team's real per-sprint capacity is closer to 13 points

than the 26 points committed each time, and future sprint planning should scale commitments down toward roughly half of what was attempted here. A third or fourth sprint would help confirm whether 13 points is a stable velocity or whether the 50%-completion pattern is itself a coincidence of how the simulation was structured.

End of submission — Lab 2: Agile Backlog Creation & Sprint Simulation in Jira.